

Name-Ayush Kumar Rai

Project Description:CI/CD Pipeline Setup with Git, Jenkins, and Kubernetes

Problem Statement: CI/CD Pipeline Setup with Git, Jenkins, and Kubernetes

Scenario:

You are tasked with setting up a Continuous Integration/Continuous Deployment (CI/CD) pipeline using Git, Jenkins, and Kubernetes (K8s). The goal is to automate the process of building artifacts upon code commits to specific Git repositories and deploying them to designated Kubernetes clusters.

Requirements:

- Whenever a commit is made to a specific Git repository, Jenkins should automatically trigger a build process.
- Upon successful build, Jenkins should deploy the artifact to a specified Kubernetes cluster.
- The solution should be scalable and adaptable to accommodate different Git repositories and Kubernetes clusters.
- Utilize Groovy scripting to implement the automation logic, ensuring flexibility and extensibility.

Expectations:

- The CI/CD pipeline should streamline the software delivery process, reducing manual intervention and ensuring consistency.
- Automation scripts should be well-documented and easily understandable for maintenance and future enhancements.
- The solution should be robust, handling errors gracefully and providing clear feedback in case of failures.
- Security best practices should be implemented throughout the pipeline to safeguard code integrity and deployment environments.

Deliverables:

- Groovy script(s) for Jenkins pipeline configuration, including stages for building artifacts and deploying to Kubernetes.
- Documentation outlining the setup instructions, including how to configure Git repositories and Kubernetes clusters for integration with the CI/CD pipeline.
- Optional: Test cases and validation procedures to ensure the correctness and reliability of the automation solution.
- Optional: Recommendations for monitoring and logging strategies to track the performance and health of the CI/CD pipeline.

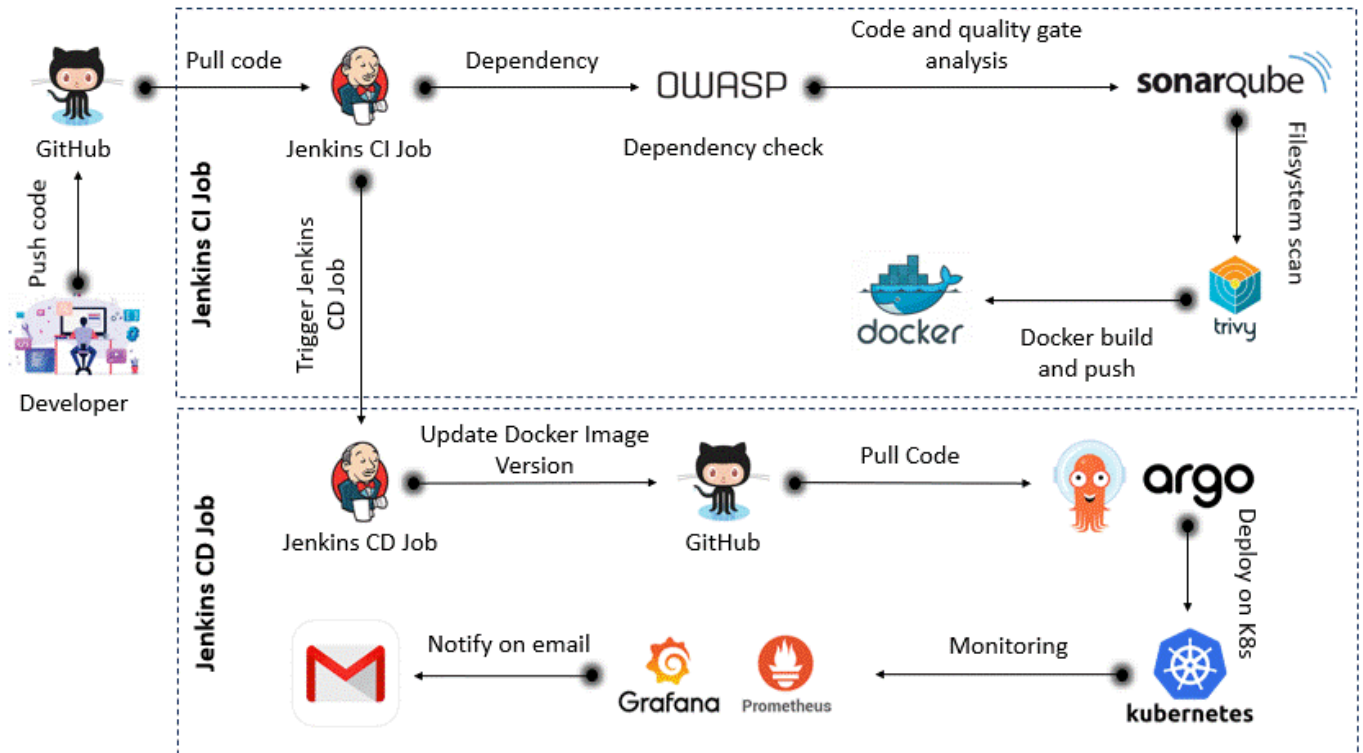
Additional Information:

- Consider utilizing Jenkins plugins for Git integration and Kubernetes deployment to leverage existing functionalities and enhance productivity.
- Follow best practices for version control and documentation to maintain the integrity and transparency of the CI/CD pipeline implementation.

GitHub Repo-<https://github.com/Ayushrai001/Wanderlust-Mega-Project-main.git>

Jenkins Shared file-https://github.com/Ayushrai001/Jenkins_SharedLib.git

◆ 1. Project Overview



This project implements a **fully automated CI/CD pipeline** that integrates:

- GitHub (Code)
- Docker (Containerization)
- Jenkins (CI)
- OWASP (Dependency check)
- SonarQube (Quality)
- Trivy (Filesystem Scan)
- ArgoCD (CD)
- Redis (Caching)
- AWS EKS (Kubernetes)
- Helm (Monitoring using grafana and prometheus)

The pipeline ensures **end-to-end automation from code commit to production deployment**.

Kubernetes Deployment Details

- Namespace: wanderlust

- Services:
 - Backend (LoadBalancer)
 - Frontend (LoadBalancer)
 - MongoDB
 - Redis
- Observations:
 - Some pods initially showed:
 - ImagePullBackOff
 - ErrImagePull
 - Later resolved successfully

Challenges Faced & Solutions

Issue	Solution
ImagePullBackOff	Fixed Docker image tagging & registry auth
ArgoCD Sync Error	Corrected manifest path & repo structure
OWASP DB Error	Updated dependency-check database
Pipeline Failures	Added retry & logging

EKS Cluster with Loadbalancer

The screenshot shows the AWS Management Console for the EC2 Instances page. The left sidebar contains navigation links for EC2, Dashboard, AWS Global View, Events, and Instances. The main content area shows a list of four instances, all of which are in the 'Running' state. The instances are named 'wanderlust-dev-nodes-Node', 'terraform-master', 'Automate', and 'wanderlust-dev-nodes-Node'. They are all using the 'm7i-flex.large' instance type. The status checks for all instances show '3/3 checks passed'. The page includes a search bar, filter buttons, and a table with columns for Name, Instance ID, Instance state, Instance type, Status check, Alarm status, and Availability Zone.

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone
wanderlust-dev-nodes-Node	i-0999a5fc8411e0468	Running	m7i-flex.large	3/3 checks passed	View alarms	us-west-2b
terraform-master	i-005ac8df5a8ee6497	Running	m7i-flex.large	3/3 checks passed	View alarms	us-west-2a
Automate	i-0ceb0d2c9f1ada4ab	Running	m7i-flex.large	3/3 checks passed	View alarms	us-west-2a
wanderlust-dev-nodes-Node	i-0027cda2d20b32eef	Running	m7i-flex.large	3/3 checks passed	View alarms	us-west-2c

CI Workflow

Validate Parameters	Workspace cleanup	Git: Code Checkout	Trivy: Filesystem scan	OWASP: Dependency check	SonarQube: Code Analysis	SonarQube: Code Quality Gates	Exporting environment variables	Backend env setup	Frontend env setup	Docker: Build Images	Docker: Push to DockerHub	Declarative: Post Actions
197ms	220ms	1s	1s	15s	29s	426ms	105ms	2s	2s	35s	5s	107ms
219ms	257ms	1s	1s	24s	23s	373ms (paused for 27s)	91ms	1s	1s	1min 42s		
158ms	170ms	737ms	667ms	8s	25s	591ms (paused for 1s)	140ms	2s	2s	1s	6s	76ms
214ms	235ms	1s	1s	14s	40s	315ms (paused for 1s)	86ms	2s	2s	1s	3s	138ms

SonarQube Quality Gate

wanderlust **Passed**

server-side processing: **Success**

Permalinks

- Last build (#21), 1 min 53 sec ago

CI fail

Wanderlust-Cl

Status

Changes

Build with Parameters

Configure

Delete Pipeline

Full Stage View

GitHub

Favorite

SonarQube

Open Blue Ocean

Stages

Rename

Pipeline Syntax

Wanderlust-Cl

this is the CI pipeline of Wanderlust

Stage View

	Validate Parameters	Workspace cleanup	Git: Code Checkout	Trivy: Filesystem scan	OWASP: Dependency check	SonarQube: Code Analysis	SonarQube: Code Quality Gates	Exporting environment variables	Backend env setup	Frontend env setup	Docker: Build Images	Docker: Push to DockerHub	Declarati Post Actions
Average stage times:	134ms	176ms	736ms	663ms	8s	23s	247ms	59ms	2s	2s	1s	5s	110ms
#17 Apr 13 10:14 commit	89ms	180ms	777ms	665ms	8s	23s	262ms (skipped for 800ms)	50ms	2s	2s	1s	3s	127ms
#16 Apr 13 10:10 No Changes	180ms	172ms	696ms	662ms	8s	24s	232ms (skipped for 1s)	69ms	2s	2s	994ms	7s	94ms

Builds >

Filter

Today

#17 4:44 am

#16 4:40 am

SonarQube Quality Gate

wanderlust Passed

server-side processing: Success

Permalinks

- Last build (#17), 16 min ago
- Last failed build (#17), 16 min ago
- Last unsuccessful build (#17), 16 min ago
- Last completed build (#17), 16 min ago

CD Builds and Fails

Jenkins

Wonderlust-CD

Status

Changes

Build with Parameters

Configure

Delete Pipeline

Full Stage View

Favorite

Open Blue Ocean

Stages

Rename

Pipeline Syntax

Builds

Filter

Today

#6 7:20 am

#5 6:00 am

Wonderlust-CD

This is the CD of Wonderlust

Stage View

	Workspace cleanup	Git: Code Checkout	Verify: Docker Image Tags	Update: Kubernetes manifests	Git: Code update and push to GitHub	Declarative: Post Actions
Average stage times: (full run time: ~5s)	193ms	999ms	138ms	818ms	993ms	1s
#6 Apr 13 12:50 3 commits	225ms	1s	119ms	823ms	897ms failed	2s
#5 Apr 13 11:30 4 commits	191ms	737ms	151ms	817ms	1s	1s
#4 Apr 13 10:15	164ms	733ms	145ms	816ms	1s	1s

Permalinks

Last build (#6), 0.32 sec ago

Edit description

Jenkins

Wonderlust-CD

#5

Stages

Started by Ayush Kumar Rai

Started 4 hr 46 min ago

Queued 3 ms

Took 5.6 sec

Parameters

Changes

Graph

Start Workspace cleanup Git: Code Checkout Verify: Docker Image... Update: Kubernetes... Git: Code update and... Post Actions End

Search

Workspace cleanup 0.16s

Git: Code Checkout 0.71s

Verify: Docker Image Tags 0.12s

Update: Kubernetes manifests 0.79s

Git: Code update and push to GitHub 1s

Post Actions 1s

Post Actions

Extended Email

Sending email to: raiayush021@gmail.com

44.243.227.149:8080

15°C Sunny

Jenkins 2.541.3

04:16 PM 13-04-2026

✓ Wonderlust-CD 5

Pipeline

Changes

Tests

Artifacts

Restart

Settings

Logout

Close

Branch: —

6s

Changes by Jenkins, ubuntu

Commit: —

5 hours ago

Started by user Ayush Kumar Rai

Start

Workspace cleanup

Git: Code Checkout

Verify: Docker Image Tags

Update: Kubernetes man...

Git: Code update and push to Git...

End

Git: Code update and push to GitHub - 1s

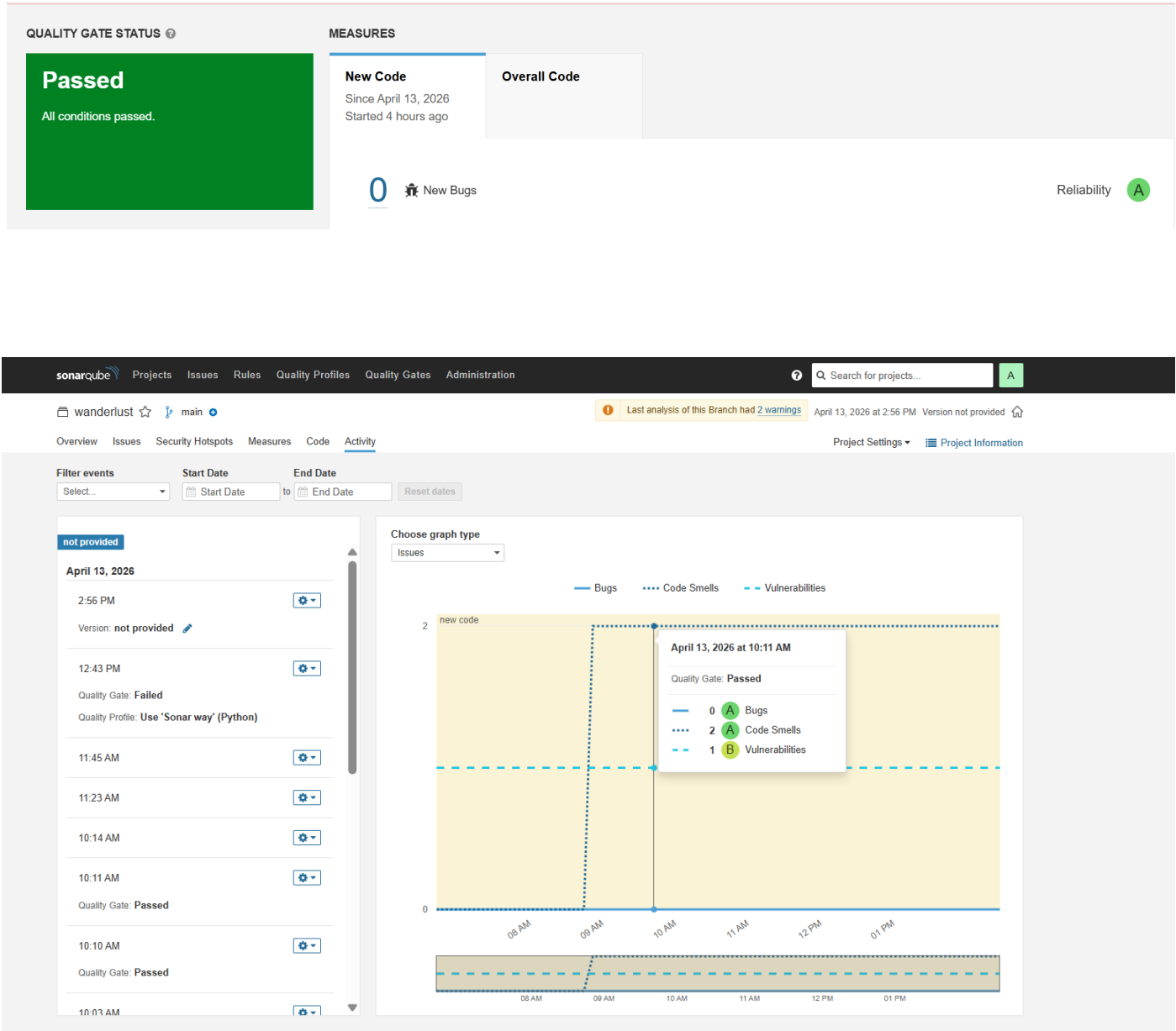
Restart Git: Code update and push to GitHub

✓ > echo "Checking repository status: " git status echo "Adding changes to git: " git add , echo "Committing changes: " git commit -m "Updated environment variables" echo "Pushing changes to github: " git push https://github.com/Ayus... — Shell Script <1s

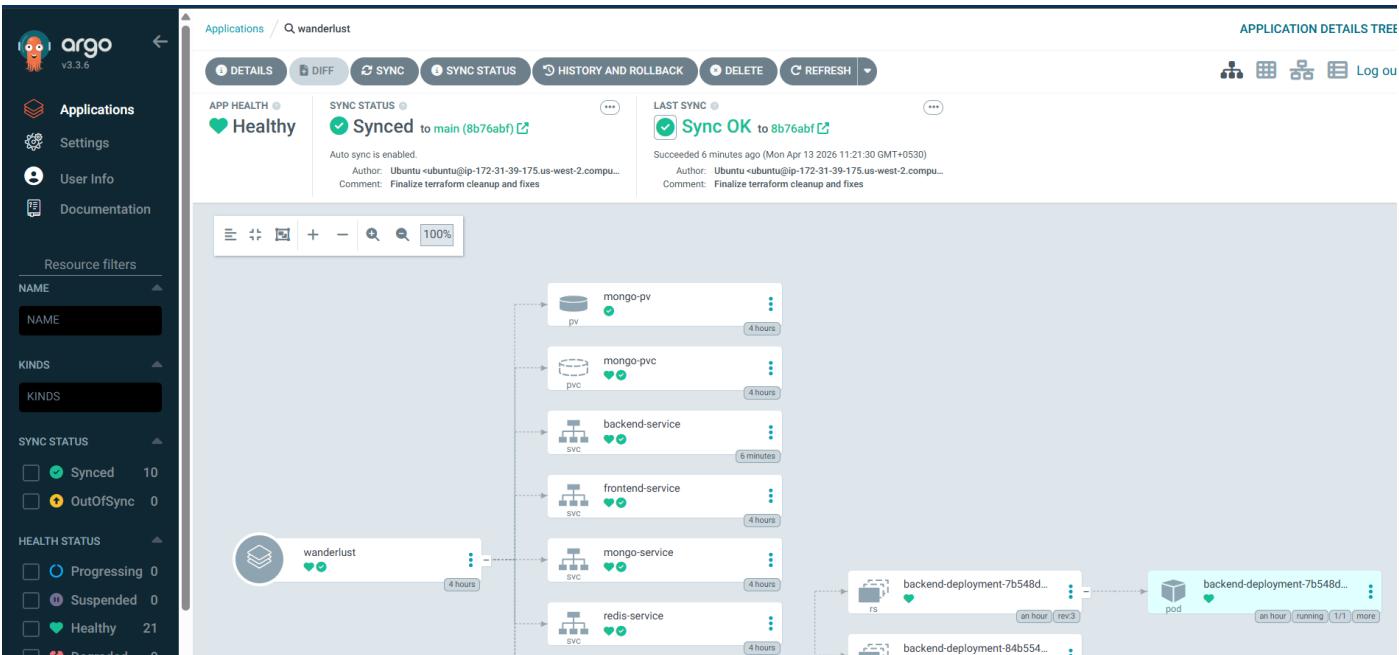
✓ v Extended Email 1s

1 Sending email to: raiayush021@gmail.com

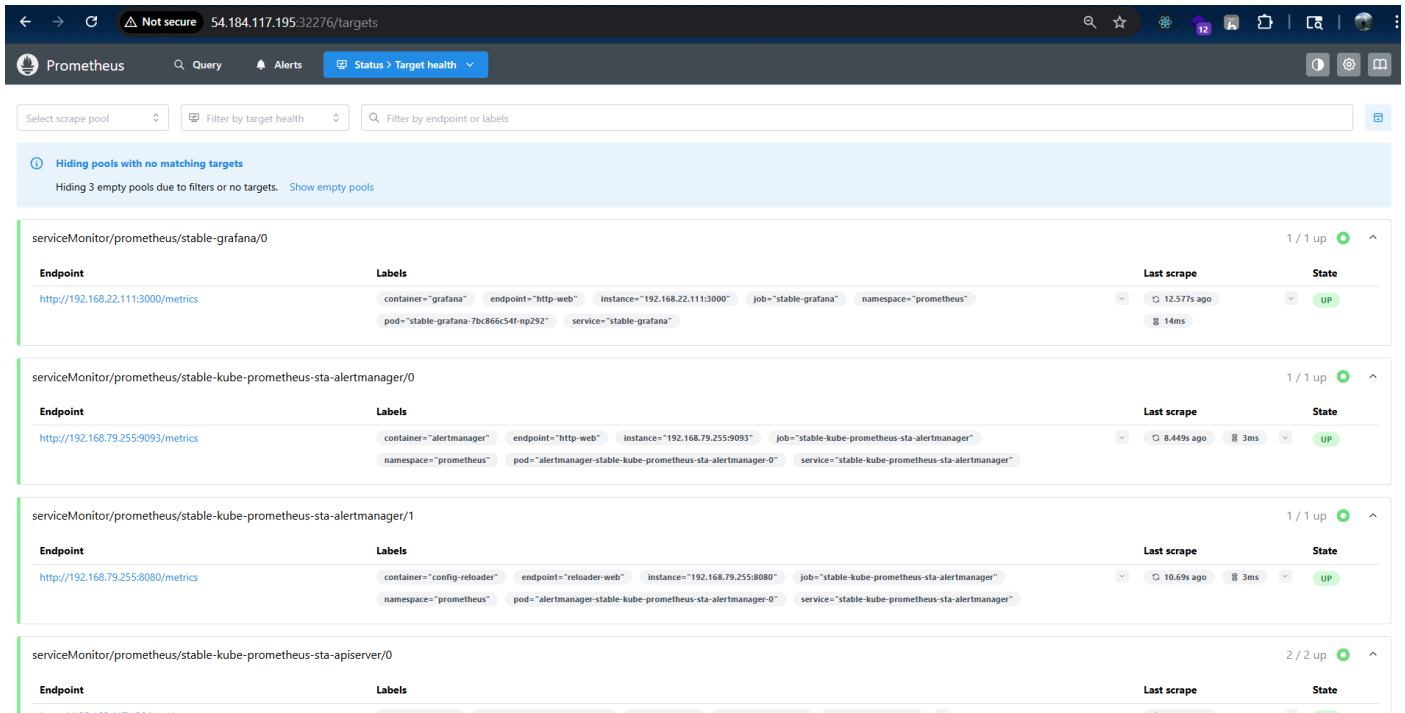
Sonar Qube



◆ 3. CD Implementation by Argo cd and K&s



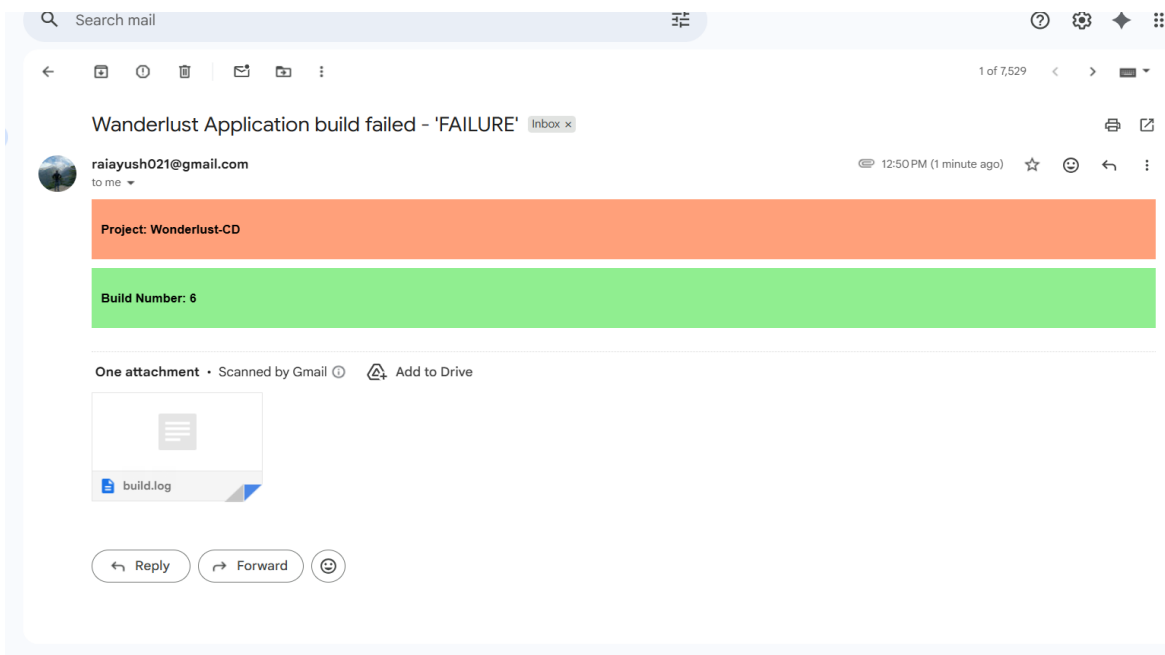
Mointoring



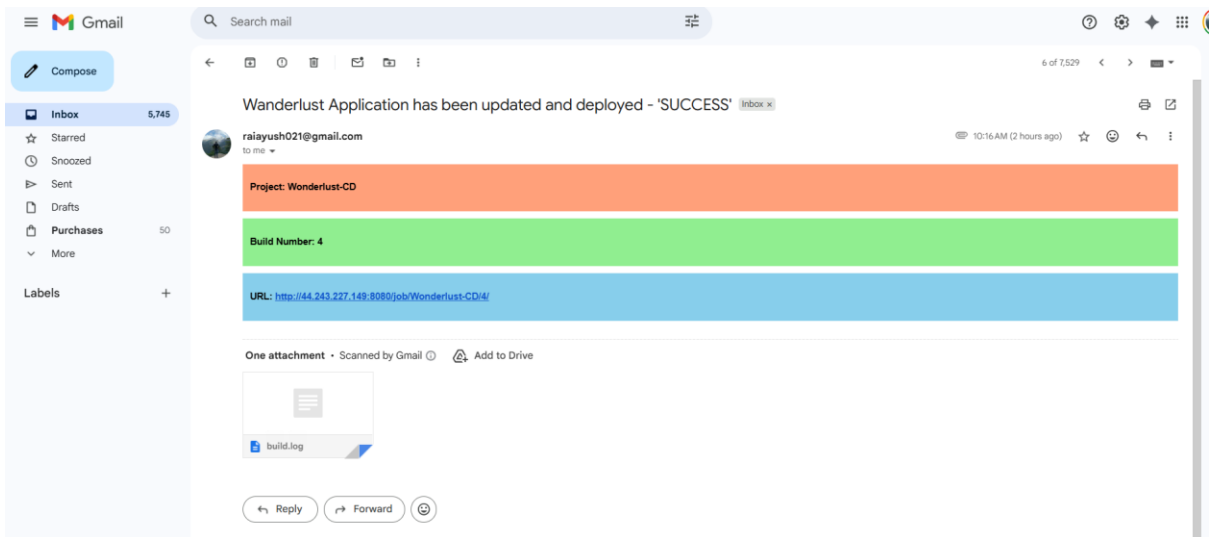
The screenshot shows the Prometheus web interface at the URL `54.184.117.195:32276/targets`. The page displays a list of targets under the 'Status > Target health' tab. A blue banner at the top indicates 'Hiding pools with no matching targets' and 'Hiding 3 empty pools due to filters or no targets. Show empty pools'. The table below lists four targets, each with its endpoint, labels, last scrape time, and state.

Endpoint	Labels	Last scrape	State
http://192.168.22.111:3000/metrics	<code>container="grafana"</code> <code>endpoint="http-web"</code> <code>instance="192.168.22.111:3000"</code> <code>job="stable-grafana"</code> <code>namespace="prometheus"</code> <code>pod="stable-grafana-7bc866c54f-np292"</code> <code>service="stable-grafana"</code>	12.577s ago 14ms	UP
http://192.168.79.255:9093/metrics	<code>container="alertmanager"</code> <code>endpoint="http-web"</code> <code>instance="192.168.79.255:9093"</code> <code>job="stable-kube-prometheus-sta-alertmanager"</code> <code>namespace="prometheus"</code> <code>pod="alertmanager-stable-kube-prometheus-sta-alertmanager-0"</code> <code>service="stable-kube-prometheus-sta-alertmanager"</code>	8.449s ago 3ms	UP
http://192.168.79.255:8080/metrics	<code>container="config-reloader"</code> <code>endpoint="reloader-web"</code> <code>instance="192.168.79.255:8080"</code> <code>job="stable-kube-prometheus-sta-alertmanager"</code> <code>namespace="prometheus"</code> <code>pod="alertmanager-stable-kube-prometheus-sta-alertmanager-0"</code> <code>service="stable-kube-prometheus-sta-alertmanager"</code>	10.69s ago 3ms	UP
[truncated]	[truncated]	[truncated]	[truncated]

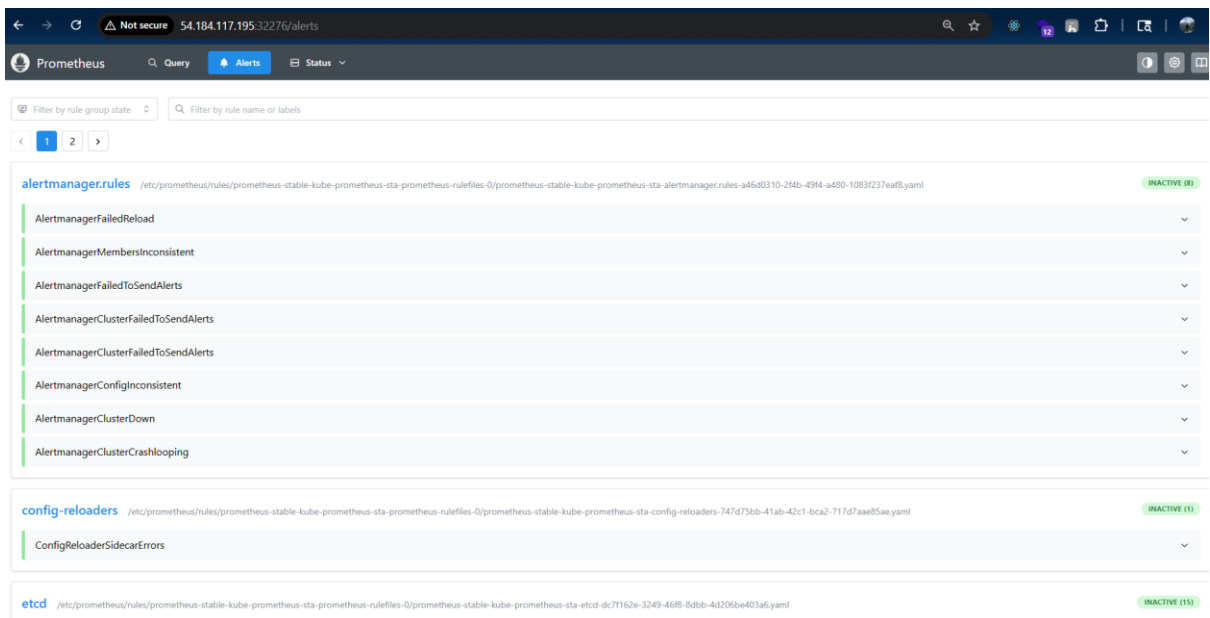
FAIL & pass CI-CD mail Notification with Log file



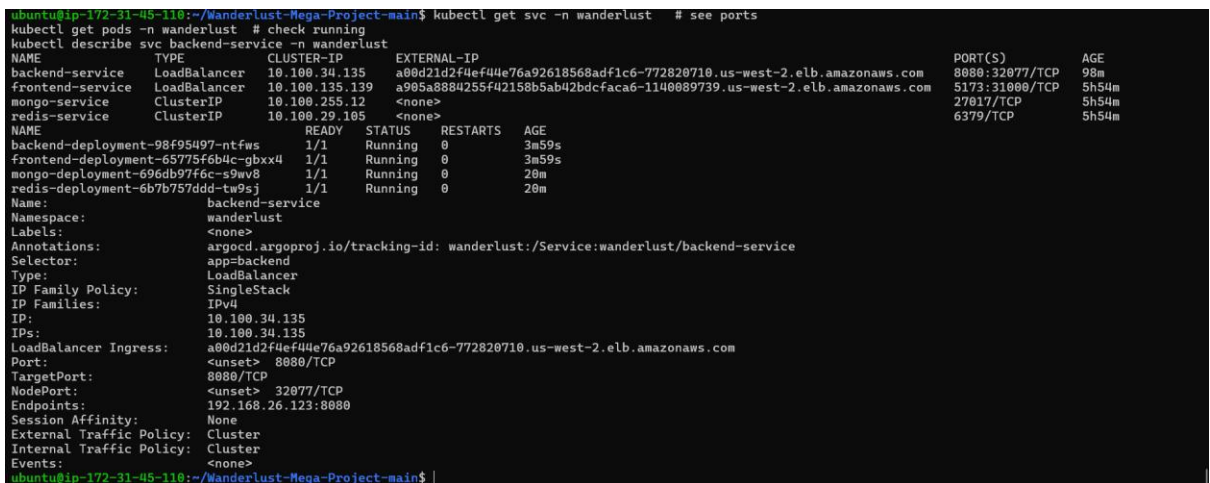
The screenshot shows a Gmail notification email titled 'Wanderlust Application build failed - FAILURE'. The email is from `raiyush021@gmail.com` and was received at 12:50 PM (1 minute ago). The email body contains two colored boxes: an orange box for 'Project: Wonderlust-CD' and a green box for 'Build Number: 6'. Below these boxes, it states 'One attachment • Scanned by Gmail' and shows a thumbnail of a file named 'build.log'. At the bottom, there are buttons for 'Reply', 'Forward', and a smiley face icon.



PROMETHEUS



K8S PODS




```
ubuntu@ip-172-31-45-110:~/Wanderlust-Mega-Project-main$ kubectl get svc -n wanderlust
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
backend-service	LoadBalancer	10.100.34.135	a00d21d2f4ef44e76a92618568adf1c6-772820710.us-west-2.elb.amazonaws.com	8080:32077/TCP	96m
frontend-service	LoadBalancer	10.100.135.139	a905a8884255f42158b5ab42bdcfaca6-1140089739.us-west-2.elb.amazonaws.com	5173:31000/TCP	5h53m
mongo-service	ClusterIP	10.100.255.12	<none>	27017/TCP	5h53m
redis-service	ClusterIP	10.100.29.105	<none>	6379/TCP	5h53m

```
ubuntu@ip-172-31-45-110:~/Wanderlust-Mega-Project-main$ |
```

```
ubuntu@ip-172-31-45-110:~/Wanderlust-Mega-Project-main$ kubectl get pods -n wanderlust
```

NAME	READY	STATUS	RESTARTS	AGE
backend-deployment-6dfb76fb9b-l6bm6	0/1	ImagePullBackOff	0	13s
backend-deployment-7b548d99d6-7wstd	1/1	Running	0	40s
frontend-deployment-5cf6569689-8jmbv	0/1	ErrImagePull	0	12s
frontend-deployment-6b64f4764d-t4l2s	1/1	Running	0	40s
mongo-deployment-696db97f6c-s9wv8	1/1	Running	0	40s
redis-deployment-6b7b757ddd-tw9sj	1/1	Running	0	40s

```
ubuntu@ip-172-31-45-110:~/Wanderlust-Mega-Project-main$ |
```